

CampusFlow Project Guide

1. What CampusFlow Is

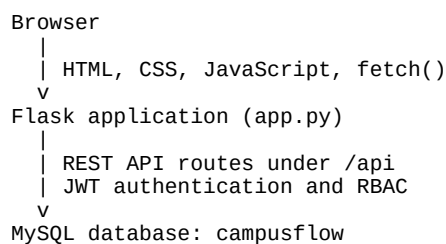
CampusFlow is a Flask-based campus management application connected to MySQL. It provides login, registration, role-based access, announcements, rooms, complaints, lost and found reports, and admin monitoring.

The application has three roles:

- ``ADMIN``: manages campus content, users, rooms, announcements, and complaints.
- ``FACULTY``: staff role. Can submit complaints and use shared campus features.
- ``STUDENT``: can submit complaints and lost-and-found reports and view permitted campus information.

The database engine is MySQL only. SQLite is not used by the application.

2. High-Level Architecture



The browser never talks directly to MySQL. The browser sends HTTP requests to Flask. Flask validates the request, checks the JWT and role, runs a parameterized SQL query, and returns JSON.

3. How the Application Starts

The recommended launcher is the root ``app.py`` file.

```
cd "C:\Users\Dell\Desktop\Campus_flow_py"
.\.venv\Scripts\Activate.ps1
python app.py
```

The terminal prints one application link:

```
http://127.0.0.1:5000/
```

Opening that link redirects to the login page. The root launcher uses the Flask app factory from ``backend.app``.

The startup sequence is:

1. Python loads the root ``app.py``.
2. ``app.py`` imports ``create_app`` from ``backend.app``.
3. Flask is created and configuration is loaded from ``.env``.

4. `CampusFlowDatabase` validates the MySQL URL.
5. The schema in `database/campusflow.sql` is checked and tables are created if missing.
6. API blueprints are registered.
7. Flask serves the frontend under `/app`.
8. The server listens on port `5000`.

4. Environment Configuration

The `.env` file contains secrets and the MySQL connection string.

```
SECRET_KEY=campusflow-secret-key-change-me
JWT_ALGORITHM=HS256
JWT_EXPIRY_MINUTES=60
DATABASE_URL=mysql+mysqlconnector://root:YOUR_PASSWORD@127.0.0.1:3306/campusflow
CORS_ORIGINS=*
```

Replace `YOUR\_PASSWORD` with the local MySQL password. The database named `campusflow` must exist before the app starts.

Create it in MySQL Workbench:

```
CREATE DATABASE campusflow;
```

Then run `database/campusflow.sql` in Workbench. The Flask app also runs the schema statements during startup.

5. Python Backend Structure

Root launcher

`app.py` is the simple command-line entry point. It creates the Flask app and starts the server.

Flask app factory

`backend/app.py` contains `create\_app()`.

It does the following:

- Creates the Flask application.
- Loads `Config`.
- Initializes MySQL.
- Registers CORS for API requests.
- Registers authentication, user, room, announcement, event, complaint, lost-and-found, notification, and analytics routes.
- Serves frontend files.
- Provides `/health` for a database health check.

Configuration

`backend/config.py` loads `.env` with `python-dotenv`. `DATABASE\_URL` must begin with:

```
mysql+mysqlconnector://
```

If another database URL is used, the application stops with a clear configuration error.

Database layer

`backend/database.py` contains the MySQL connection wrapper and application data functions.

The wrapper uses `mysql.connector`. It creates dictionary cursors so rows can be accessed using names such as `row['email']`.

All SQL values use MySQL Connector placeholders:

```
cursor = db.execute(
    'SELECT * FROM users WHERE email = %s',
    (email,)
)
```

Never build SQL by concatenating user input.

The database layer also converts MySQL date, time, datetime, and duration values into JSON-safe values before Flask returns them.

6. MySQL Tables

The schema is in `database/campusflow.sql`.

- `users`: name, email, bcrypt password, role, phone, timestamps.
- `rooms`: room number, building, floor, type, capacity, facilities, status.
- `announcements`: published campus messages and creator.
- `complaints`: complaint details, location, evidence, status, owner.
- `lost\_found`: lost or found item details, dates, contact, evidence, owner.

Primary keys use MySQL `INT AUTO\_INCREMENT`.

7. Frontend Structure

The frontend is plain HTML, CSS, and JavaScript. It does not require Node.js or a frontend build step.

HTML pages

- `frontend/login.html`: login screen.
- `frontend/register.html`: account creation screen.
- `frontend/student-dashboard.html`: student overview.

- ``frontend/faculty-dashboard.html``: faculty overview.
- ``frontend/admin-dashboard.html``: admin monitoring and management.
- ``frontend/announcements.html``: announcements and admin publishing form.
- ``frontend/rooms.html``: room list and admin room form.
- ``frontend/complaints.html``: complaint form and complaint list.
- ``frontend/lost-found.html``: lost-and-found form and list.

JavaScript files

- ``frontend/js/login.js``: sends login credentials and stores the JWT.
- ``frontend/js/register.js``: sends account details to the registration API.
- ``frontend/js/auth.js``: protects pages, changes dashboard links by role, handles logout, and displays the user name and role.
- ``frontend/js/dashboard.js``: loads live dashboard metrics and admin user/complaint data.
- Other page scripts load and submit page-specific records.

CSS files

- ``frontend/css/style.css``: login, registration, landing, colors, mirrored auth layout, and public form styles.
- ``frontend/css/dashboard.css``: sidebar, dashboards, management forms, cards, profile badge, animations, and responsive layout.
- ``frontend/css/responsive.css``: small-screen layout adjustments.

8. Login Flow

1. The user opens ``http://127.0.0.1:5000/``.
2. Flask redirects to ``/app/login.html``.
3. The user submits email and password.
4. ``login.js`` sends ``POST /api/auth/login``.
5. Flask finds the user by email.
6. The password is checked with bcrypt for new records.
7. Legacy Werkzeug hashes are also supported to prevent ``Invalid salt`` errors.
8. Flask creates a signed JWT containing the user ID, role, and expiration time.
9. The browser stores ``campusflow_token`` and ``campusflow_user`` in local storage.
10. The user is redirected to the dashboard for their role.

9. Password Security

New registrations use bcrypt:

```
hashed_password = bcrypt.hashpw(
```

```
password.encode('utf-8'),
bcrypt.gensalt()
).decode('utf-8')
```

Passwords are never stored as plain text for new accounts. The dependency is listed in `backend/requirements.txt`:

```
bcrypt==4.2.0
```

Login detects bcrypt hashes beginning with `\$2a\$`, `\$2b\$`, or `\$2y\$`. Older Werkzeug hashes are checked with Werkzeug compatibility logic.

10. Authentication and Roles

A JWT is sent in the request header:

```
Authorization: Bearer <token>
```

The backend middleware does this:

1. Reads the `Authorization` header.
2. Decodes and validates the JWT.
3. Loads the current user from MySQL.
4. Stores the user in Flask request context.
5. Applies the role decorator when a route is restricted.

The role decorator returns HTTP `403` when a user lacks permission.

11. Permission Matrix

Feature Student Faculty Admin

--- ---: ---: ---:

Register/login Yes Yes Yes

Submit complaint Yes Yes No

View own complaints Yes Yes No

View all complaints No No Yes

Submit lost-and-found report Yes Yes Yes

View lost-and-found Yes Yes Yes

View announcements Authenticated users Authenticated users Yes

Create announcements No No Yes

View rooms Authenticated users Authenticated users Yes

Add rooms No No Yes

View users No No Yes

Delete users No No Yes

Delete own admin account No No Blocked

12. Complaint Workflow

1. Student or faculty opens Complaints.
2. They enter category, title, details, location, and optional evidence.
3. The browser sends `POST /api/complaints` with the JWT.
4. The backend checks that the role is `STUDENT` or `FACULTY`.
5. The complaint is saved with the current user ID.
6. Students and faculty load their own records through `/api/complaints/my`.
7. Admins load all records through `/api/complaints`.
8. Admin results include submitter name, email, role, location, and status.

13. File and Image Uploads

Complaint and lost-and-found forms accept images, PDF, DOC, and DOCX files.

The browser reads the selected file with `FileReader`, converts it to a data URL, and sends it in the `image` field. The existing MySQL `TEXT` column stores that evidence value.

The browser rejects files larger than 5 MB.

This approach is convenient for local development. For production, large files should be stored in object storage or a dedicated file service, with only a secure file URL stored in MySQL.

14. Admin User Deletion

The admin dashboard loads `/api/users` and displays a Delete button for each user.

When clicked:

1. The browser asks for confirmation.
2. It sends `DELETE /api/users/<user_id>`.
3. The backend verifies the current role is `ADMIN`.
4. The backend blocks deleting the current admin account.
5. Personal complaints and lost-and-found records are deleted.
6. Shared announcements remain, with creator detached.
7. The user record is deleted.
8. The admin dashboard refreshes.

15. Important HTTP Endpoints

Public endpoints

- `POST /api/auth/register`
- `POST /api/auth/login`
- `POST /api/auth/logout`
- `GET /health`

User endpoints

- `GET /api/users/me`
- `GET /api/users` admin only
- `DELETE /api/users/<id>` admin only

Campus content endpoints

- `GET /api/announcements`
- `POST /api/announcements` admin only
- `GET /api/rooms`
- `POST /api/rooms` admin only
- `GET /api/complaints/my` student/faculty
- `POST /api/complaints` student/faculty
- `GET /api/complaints` admin only
- `GET /api/lost-found` authenticated users
- `POST /api/lost-found` authenticated users
- `GET /api/analytics` admin only

16. Installing Dependencies

Use the project virtual environment:

```
cd "C:\Users\Dell\Desktop\Campus_flow_py"
.\.venv\Scripts\Activate.ps1
python -m pip install -r backend/requirements.txt
```

The important packages are:

- Flask: web server and routing.
- Flask-Cors: browser API access.
- PyJWT: JWT creation and validation.
- bcrypt: password hashing.
- python-dotenv: `.env` loading.
- mysql-connector-python: MySQL access.
- pytest: backend tests.

17. Testing

Run the API tests with:

```
.\.venv\Scripts\python.exe -m pytest backend/tests/test_api.py -q
```

The tests cover registration, login, current-user access, student denial of admin analytics, and admin announcement creation.

The tests use the configured MySQL database and generate unique email addresses. They create test records, so reset the database afterward if you want a completely empty dashboard.

18. Common Problems

`ModuleNotFoundError: No module named 'flask'`

The active virtual environment does not have the dependencies. Run:

```
.\.venv\Scripts\python.exe -m pip install -r backend/requirements.txt
```

`ValueError: Invalid salt`

This means an old non-bcrypt hash was passed to bcrypt. The current login code detects legacy Werkzeug hashes and checks them with the compatible verifier.

MySQL error 1064

Make sure the schema uses MySQL syntax such as:

```
INT AUTO_INCREMENT PRIMARY KEY
```

Do not use SQLite syntax such as `AUTOINCREMENT`.

Database connection error

Check that:

- MySQL Server is running.
- The ``campusflow`` database exists.
- The password in ``.env`` is correct.
- MySQL is listening on port ``3306``.
- ``mysql-connector-python`` is installed in the active ``.venv``.

Port 5000 is already in use

Stop the previous Flask process with ``Ctrl+C``, then run ``python app.py`` again.

Browser shows old CSS or JavaScript

Use `Ctrl+F5` to force refresh. A `304` response is normal and means the browser reused a cached file.

19. Typical User Journey

Student

1. Register as `STUDENT`.
2. Log in.
3. See the student dashboard and live counts.
4. Submit a complaint or lost-and-found report.
5. Track personal complaints and lost-and-found reports.

Faculty

1. Register as `FACULTY`.
2. Log in.
3. Create an event or notification.
4. Submit a complaint or lost-and-found report.
5. View permitted campus information.

Admin

1. Register as `ADMIN`.
2. Log in.
3. View structured monitoring metrics.
4. Publish announcements.
5. Add rooms.
6. Submit complaints and lost-and-found reports.
7. View all users and complaints.
8. Delete registered users when necessary.

20. Development Notes

The root launcher is intentionally simple. The backend owns API behavior, authentication, database access, and authorization. The frontend owns layout, forms, local token storage, and navigation.

For production deployment, use a production WSGI server, HTTPS, secure secrets, a real file-storage service, stricter CORS, and an admin-approval workflow for administrator accounts.